Module 04: Linux Fundamentals

Learning Objectives

By the end of this module, you will be able to:

- Navigate the Linux file system from the terminal
- Create, move, copy, and delete files and directories
- Understand and modify file permissions
- Manage environment variables
- Monitor and manage processes
- Install packages using a package manager
- Feel comfortable living in the terminal

4.1 Why Linux?

Every production server you will ever deploy to runs Linux. AWS, GCP, Azure, Docker containers — all Linux. If you cannot navigate a Linux terminal, you cannot debug production issues.

macOS is Unix-based and shares most commands with Linux. If you're on macOS, everything here applies with minor differences. If you're on Windows, install WSL2 now.

4.2 The File System

Structure

```
/                       ← Root. Everything starts here.
├── bin/                ← Essential binaries (ls, cp, mv)
├── etc/                ← Configuration files
├── home/               ← User home directories
│   └── devak/          ← Your home directory (~)
│       ├── Documents/
│       ├── projects/
│       └── .bashrc     ← Shell configuration (hidden file)
├── var/                ← Variable data (logs, databases)
│   └── log/            ← System logs
├── tmp/                ← Temporary files (cleared on reboot)
├── usr/                ← User programs
│   ├── bin/            ← Non-essential binaries
│   └── local/          ← Locally installed software
├── opt/                ← Optional/third-party software
└── dev/                ← Device files
```

Key Concepts

- **Everything is a file.** Directories, devices, pipes — all files in Linux.

- **Hidden files** start with a dot: `.bashrc`, `.gitignore`, `.env`
- **Home directory** is `~` (shorthand for `/home/yourusername`)
- **Root** means two things: the `/` directory AND the `root` superuser

Absolute vs Relative Paths

```
Absolute: starts from root /
  /home/devak/projects/myapp/main.py

Relative: starts from current directory
  projects/myapp/main.py      (from /home/devak/)
  ./main.py                   (from /home/devak/projects/myapp/)
  ../myapp/main.py            (from /home/devak/projects/other/)

Special paths:
  .   → current directory
  ..  → parent directory
  ~   → home directory
  -   → previous directory (for cd only)
```

4.3 Command Mastery

Every command below includes: what it does, syntax, example, exercise, and a "try breaking it" experiment.

`pwd` — Print Working Directory

What it does: Shows your current location in the file system.

Syntax: `pwd`

Example:

```
$ pwd
/home/devak/projects
```

Exercise: Run `pwd` from three different directories. Notice how the output changes.

Try breaking it: You can't break `pwd`. It always works. But try: what does `PWD` (uppercase) do?

`ls` — List Directory Contents

What it does: Shows files and directories in the current (or specified) directory.

Syntax: `ls [options] [directory]`

Examples:

```
ls                  # List current directory
ls -l               # Long format (permissions, size, date)
ls -a               # Show hidden files (dotfiles)
ls -la              # Long format + hidden files
ls -lh              # Long format with human-readable sizes
ls -lt              # Sort by modification time
ls -R               # Recursive (show subdirectories)
ls /etc             # List a specific directory
```

Reading `ls -l` output:

```
drwxr-xr-x  5 devak staff  160 Jan 15 10:30 projects
-rw-r--r--  1 devak staff  420 Jan 14 09:15 notes.txt
│├─┤├─┤├─┤  │  │     │     │    │             │
│ │  │  │   │  │     │     │    │             └── Name
│ │  │  │   │  │     │     │    └── Modification date
│ │  │  │   │  │     │     └── Size in bytes
│ │  │  │   │  │     └── Group
│ │  │  │   │  └── Owner
│ │  │  │   └── Link count
│ │  │  └── Others permissions
│ │  └── Group permissions
│ └── Owner permissions
└── Type (d=directory, -=file, l=symlink)
```

Exercise: List the contents of your home directory in long format with hidden files. Identify three hidden files and look up what they do.

Try breaking it: Run `ls /nonexistent`. What error do you get? Run `ls -Z`. What happens with an invalid flag?

`cd` — Change Directory

What it does: Moves you to a different directory.

Syntax: `cd [directory]`

Examples:

```
cd /home/devak/projects   # Absolute path
cd projects               # Relative path
cd ..                     # Go up one level
cd ../..                  # Go up two levels
cd ~                      # Go to home directory
cd                        # Also goes to home directory
cd -                      # Go to previous directory
```

Exercise: Starting from your home directory, navigate to `/tmp`, then to `/etc`, then back to your home directory using `cd -` and `cd ~`.

Try breaking it: Run `cd /root`. What happens? Why? Run `cd somefile.txt` (where somefile.txt is a file, not a directory).

`mkdir` — Make Directory

What it does: Creates a new directory.

Syntax: `mkdir [options] directory_name`

Examples:

```
mkdir myproject                    # Create a directory
mkdir -p myproject/src/utils       # Create nested directories (-p =
parents)
mkdir dir1 dir2 dir3              # Create multiple directories
```

Exercise: Create this directory structure in one command:

```
practice/
├── src/
│   ├── models/
│   └── routes/
├── tests/
└── docs/
```

Try breaking it: Run `mkdir myproject` twice. What happens? Run `mkdir /etc/mydir`. What happens and why?

`touch` — Create Empty File / Update Timestamp

What it does: Creates an empty file if it doesn't exist, or updates the modification timestamp if it does.

Syntax: `touch filename`

Examples:

```
touch newfile.txt           # Create empty file
touch file1.py file2.py     # Create multiple files
touch existing_file.txt     # Update timestamp (doesn't erase contents)
```

Exercise: Create 5 Python files in a directory. Then use `ls -lt` to see them sorted by time. Touch the oldest one. Check the order again.

Try breaking it: Run `touch /etc/testfile`. What happens? Why?

`rm` — Remove Files and Directories

What it does: Deletes files or directories. **There is no trash can. Deleted = gone.**

Syntax: `rm [options] file`

Examples:

```
rm file.txt                 # Delete a file
rm -i file.txt              # Ask for confirmation
rm -r directory/            # Delete directory and all contents
rm -rf directory/           # Force delete, no confirmation
```

Exercise: Create a directory with 3 files inside. Delete one file. Then delete the entire directory.

Try breaking it: Run `rm` with no arguments. What happens? **DANGER: Never run `rm -rf /` or `rm -rf ~`. This destroys your system or home directory. Understand what `-rf` does before using it.**

`cp` — Copy Files and Directories

What it does: Copies files or directories.

Syntax: `cp [options] source destination`

Examples:

```
cp file.txt backup.txt          # Copy file
cp file.txt ../                 # Copy to parent directory
cp -r src/ src_backup/          # Copy directory recursively
cp file1.txt file2.txt dest/    # Copy multiple files to directory
```

Exercise: Create a file with some content. Copy it to a new name. Verify both files exist and have the same content.

Try breaking it: Run `cp file.txt file.txt`. What happens? Run `cp -r dir1/ dir1/subdir/`. What happens?

`mv` — Move/Rename Files

What it does: Moves files to a new location OR renames them (same operation).

Syntax: `mv source destination`

Examples:

```
mv old_name.txt new_name.txt    # Rename
mv file.txt ../                 # Move to parent directory
mv file.txt ~/projects/         # Move to specific directory
mv dir1/ dir2/                  # Rename directory
```

Exercise: Create a file called `draft.txt`. Rename it to `final.txt`. Move it to a subdirectory.

Try breaking it: Run `mv file.txt file.txt`. Run `mv nonexistent.txt somewhere/`.

`cat` — Concatenate and Display

What it does: Displays file contents. Can also concatenate multiple files.

Syntax: `cat [file...]`

Examples:

```
cat file.txt                    # Display file contents
cat file1.txt file2.txt         # Display multiple files
cat file.txt | head -5          # First 5 lines
cat file.txt | tail -5          # Last 5 lines
cat > newfile.txt               # Create file (type content, Ctrl+D to
save)
cat file1.txt file2.txt > combined.txt  # Concatenate files
```

Exercise: Create two text files with different content. Use `cat` to combine them into a third file.

Try breaking it: Run `cat /dev/urandom`. Press Ctrl+C to stop. What happened?

`nano` — Terminal Text Editor

What it does: Simple terminal-based text editor.

Syntax: `nano [filename]`

Key commands:

```
Ctrl+O    → Save (Write Out)
Ctrl+X    → Exit
Ctrl+K    → Cut line
Ctrl+U    → Paste line
Ctrl+W    → Search
Ctrl+G    → Help
```

Exercise: Open a new file with nano, write a Python hello world program, save it, exit, and run it with `python3`.

Try breaking it: Open a file you don't have permission to write. Try saving. What happens?

`clear` — Clear Terminal Screen

What it does: Clears the terminal display. (Content is not deleted, just scrolled up.)

Syntax: `clear` or `Ctrl+L`

4.4 File Permissions

Every file has three permission sets: **owner**, **group**, **others**.

Each set has three permissions: **read (r)**, **write (w)**, **execute (x)**.

```
-rwxr-xr--
|||||||||
|||||||||└── others: read
||||||||└─── others: no write
|||||||└──── others: no execute... wait, that's wrong.
```

Let me lay this out clearly:

```
-  rwx  r-x  r--
|  |||  |||  |||
|  |||  |||  ||└── others: no execute
|  |||  |||  |└─── others: no write
|  |||  |||  └──── others: read
|  |||  ||└─────── group: execute
|  |||  |└──────── group: no write
|  |||  └───────── group: read
|  ||└────────────── owner: execute
|  |└─────────────── owner: write
|  └──────────────── owner: read
└───────────────────── type (- = file, d = directory)
```

Numeric (Octal) Notation

```
r = 4
w = 2
x = 1

rwx = 4+2+1 = 7
r-x = 4+0+1 = 5
r-- = 4+0+0 = 4

So -rwxr-xr-- = 754
```

`chmod` — Change Permissions

```
chmod 755 script.sh       # rwxr-xr-x (common for scripts)
chmod 644 file.txt        # rw-r--r-- (common for regular files)
chmod 600 secrets.env     # rw------- (only owner can read/write)
chmod +x script.sh        # Add execute for all
chmod u+x script.sh       # Add execute for owner only
chmod go-w file.txt       # Remove write for group and others
```

`sudo` — Superuser Do

```
sudo apt update            # Run as root
sudo nano /etc/hosts       # Edit system file as root
sudo !!                    # Re-run last command as root
```

`sudo` gives you root privileges. Use it only when necessary. If a command fails with "Permission denied," think about whether you actually need root before blindly adding `sudo`.

Exercise 4.1: Permission Practice

```
# Create a file and a script
echo "hello" > myfile.txt
echo '#!/bin/bash\necho "Running!"' > myscript.sh

# Check default permissions
ls -l myfile.txt myscript.sh

# Try to execute the script
./myscript.sh    # Will fail — no execute permission

# Fix it
chmod +x myscript.sh
./myscript.sh    # Works now

# Make a file readable only by you
chmod 600 myfile.txt
ls -l myfile.txt

# Try to read it as another user (if available)
# Or observe that group/others have no permissions
```

4.5 Environment Variables

Environment variables are key-value pairs available to all processes in a shell session.

Viewing

```
env                         # Show all environment variables
echo $HOME                  # Show specific variable
echo $PATH                  # Show PATH
echo $USER                  # Current user
echo $SHELL                 # Current shell
printenv HOME               # Alternative way
```

Setting

```
# For current session only
export MY_VAR="hello"
echo $MY_VAR                # "hello"

# For a single command
DATABASE_URL="postgres://..." python3 app.py

# Permanent (add to ~/.bashrc or ~/.zshrc)
echo 'export MY_VAR="hello"' >> ~/.zshrc
source ~/.zshrc              # Reload
```

The PATH Variable

PATH tells the shell where to find executables.

```
echo $PATH
# /usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin

# The shell searches these directories in order when you type a command.
# "ls" → found in /bin/ls
# "python3" → found in /usr/local/bin/python3

# Add a directory to PATH
export PATH="$HOME/.local/bin:$PATH"

# Find where a command lives
which python3               # /usr/local/bin/python3
which ls                    # /bin/ls
```

.env Files

For application configuration, use `.env` files:

```
# .env
DATABASE_URL=postgres://user:pass@localhost:5432/mydb
SECRET_KEY=mysecretkey123
DEBUG=true
```

Never commit `.env` files to Git. They contain secrets.

Exercise 4.2: Environment Variables

```
# 1. Set a variable and print it
export GREETING="Hello from the terminal"
echo $GREETING

# 2. Use it in a Python script
python3 -c "import os; print(os.environ.get('GREETING', 'not set'))"

# 3. Unset it
unset GREETING
echo $GREETING    # Empty

# 4. Check your PATH
echo $PATH | tr ':' '\n'    # One directory per line
```

4.6 Process Management

A process is a running program. Every command you run is a process.

Viewing Processes

```
ps                      # Your processes
ps aux                  # All processes on the system
ps aux | grep python    # Find Python processes

top                     # Live process viewer (q to quit)
htop                    # Better live viewer (install: brew install htop)
```

Reading `ps aux` Output

```
USER     PID  %CPU %MEM    VSZ   RSS TTY   STAT  TIME COMMAND
devak   1234  0.5  1.2  123456 12345 pts/0  S    0:01 python3 app.py
|       |     |    |                        |
|       |     |    |                        └── Process state
|       |     |    └── Memory usage %
|       |     └── CPU usage %
```

```
│       └── Process ID (PID)
└── Owner
```

Process States

```
R = Running
S = Sleeping (waiting for something)
D = Uninterruptible sleep (waiting for I/O)
Z = Zombie (finished but parent hasn't collected exit status)
T = Stopped
```

Killing Processes

```
kill 1234              # Send SIGTERM (graceful shutdown) to PID 1234
kill -9 1234           # Send SIGKILL (force kill) — last resort
killall python3        # Kill all python3 processes

# Common signals
# SIGTERM (15) — "Please stop" — process can clean up
# SIGKILL (9)  — "Stop now" — process cannot catch this
# SIGINT (2)   — Ctrl+C — interrupt
```

Background and Foreground

```
python3 server.py &    # Run in background
jobs                   # List background jobs
fg %1                  # Bring job 1 to foreground
bg %1                  # Resume stopped job in background
Ctrl+Z                 # Suspend (stop) current foreground process
Ctrl+C                 # Interrupt (kill) current foreground process
```

Exercise 4.3: Process Management

```
# 1. Start a long-running process
sleep 300 &

# 2. Find it
ps aux | grep sleep

# 3. Note the PID
jobs -l

# 4. Kill it gracefully
kill %1
```

```
# 5. Start a Python server in the background
python3 -m http.server 9000 &

# 6. Verify it's running
curl http://localhost:9000

# 7. Find and kill it
lsof -i :9000
kill $(lsof -t -i :9000)
```

4.7 Installing Packages

Debian/Ubuntu (apt)

```
sudo apt update                     # Update package list
sudo apt upgrade                    # Upgrade installed packages
sudo apt install nginx              # Install a package
sudo apt remove nginx               # Remove a package
apt search postgresql               # Search for packages
```

macOS (brew)

```
brew update                         # Update Homebrew
brew install nginx                  # Install a package
brew uninstall nginx                # Remove a package
brew search postgres                # Search for packages
brew list                           # List installed packages
```

Python (pip)

```
pip install fastapi                 # Install Python package
pip install -r requirements.txt     # Install from requirements file
pip freeze                          # List installed packages with
versions
pip uninstall fastapi               # Remove package
```

4.8 Essential Commands — Advanced

I/O Redirection

```
echo "hello" > file.txt            # Write to file (overwrite)
echo "world" >> file.txt           # Append to file
cat nonexistent 2> errors.txt      # Redirect stderr
command > out.txt 2>&1             # Redirect both stdout and stderr
```

Pipes

```
# Pipe sends output of one command as input to another
cat /var/log/syslog | grep error
ps aux | grep python | wc -l       # Count Python processes
history | grep git                  # Find git commands in history
```

Useful Utilities

```
wc -l file.txt                     # Count lines
sort file.txt                      # Sort lines
uniq                               # Remove adjacent duplicates
head -20 file.txt                  # First 20 lines
tail -20 file.txt                  # Last 20 lines
tail -f /var/log/syslog            # Follow log in real time
grep "pattern" file.txt            # Search in file
grep -r "pattern" directory/       # Search recursively
find . -name "*.py"                # Find files by name
find . -name "*.log" -mtime +7     # Find logs older than 7 days
which python3                      # Where is this command?
whoami                             # Current user
hostname                           # Machine name
df -h                              # Disk usage
du -sh directory/                  # Directory size
curl -O https://example.com/file   # Download file
tar -czf archive.tar.gz dir/       # Create compressed archive
tar -xzf archive.tar.gz            # Extract archive
```

Checkpoint Quiz

1. What is the difference between `/home/devak/file.txt` and `./file.txt`?
2. What does `chmod 600 file.txt` do?
3. What is the PATH variable? Why does it matter?
4. How do you kill a process with PID 5432?
5. What does `>` do differently from `>>`?
6. What does `ps aux | grep python` do?
7. Why should you never run `rm -rf /`?
8. What does `sudo` do? When should you use it?
9. How do you set an environment variable for just one command?

10. What does `.` mean as a path? What about `..`?

Common Mistakes

1. **Using `sudo` for everything.** Only use it when you actually need root permissions. Running pip with sudo can corrupt your Python installation.
2. **Not using `-p` with `mkdir`.** `mkdir a/b/c` fails if `a/b` doesn't exist. `mkdir -p a/b/c` creates the whole chain.
3. **Forgetting `-r` with `cp` and `rm` for directories.** `cp dir1/ dir2/` doesn't work without `-r`.
4. **Editing `.bashrc` but not sourcing it.** Changes don't take effect until you run `source ~/.bashrc` or open a new terminal.
5. **Putting spaces around `=` in variable assignment.** `MY_VAR = "hello"` is wrong. `MY_VAR="hello"` is correct.
6. **Not quoting variables.** `rm $FILE` is dangerous if FILE contains spaces. Use `rm "$FILE"`.
7. **Ignoring error messages.** Read them. They usually tell you exactly what's wrong.

Mini Project: System Information Script

Create a bash script that displays system information:

```bash
#!/bin/bash
# sysinfo.sh

echo "=== System Information ===="
echo "Hostname: $(hostname)"
echo "User: $(whoami)"
echo "Date: $(date)"
echo "Uptime: $(uptime)"
echo ""
echo "=== Disk Usage ===="
df -h | head -5
echo ""
echo "=== Memory ===="
free -h 2>/dev/null || vm_stat 2>/dev/null  # Linux vs macOS
echo ""
echo "=== Top 5 Processes (by CPU) ===="
ps aux --sort=-%cpu 2>/dev/null | head -6 || ps aux -r | head -6
echo ""
echo "=== Network ===="
echo "IP: $(hostname -I 2>/dev/null || ipconfig getifaddr en0
2>/dev/null)"
echo "Open ports:"
ss -tlnp 2>/dev/null | head -10 || lsof -i -P | grep LISTEN | head -10
echo ""
echo "=== Python ===="
echo "Python: $(python3 --version 2>/dev/null || echo 'not installed')"
echo "Pip: $(pip3 --version 2>/dev/null || echo 'not installed')"
echo "=== Git ===="
echo "Git: $(git --version 2>/dev/null || echo 'not installed')"
```

```
# Make it executable and run it
chmod +x sysinfo.sh
./sysinfo.sh
```

Next Module

Proceed to `05_git_and_version_control.md` →